

Resumo Extenso: Módulo 3 - Branches, Colaboração e Infraestrutura Git

Arquitetura de Branches e HEAD

- Uma branch não é uma cópia completa dos arquivos do projeto, mas sim uma referência leve (um ponteiro) nomeada que aponta para um commit específico.
- A branch **main** (ou **master**) é uma convenção de governança que representa a linha de desenvolvimento principal e estável do projeto.
- O ponteiro **HEAD** indica o contexto atual de trabalho; em estado normal, ele aponta para uma branch, que por sua vez aponta para um commit.
- O estado *detached HEAD* ocorre quando o **HEAD** deixa de apontar para uma branch e passa a apontar diretamente para um commit específico, sendo útil para inspecionar versões antigas.

Integração de Código: Merge vs. Rebase

- **Merge:** É a forma clássica de integração.
 - *Fast-forward:* Ocorre quando a branch de destino não avançou desde a criação da origem, permitindo que o Git apenas mova a referência da branch para o commit mais recente.
 - *Merge de três vias:* Ocorre quando ambas as branches avançaram; o Git cria um novo "commit de merge" que combina as alterações e possui dois pais no histórico.
- **Rebase:** Reorganiza o histórico reaplicando os commits de uma branch sobre um novo ponto de partida, criando uma linha de desenvolvimento linear.
 - A operação recria os commits, gerando novos *hashes* (reescrita de histórico).
 - O rebase interativo (**-i**) permite combinar pequenos commits (*squash*), alterar mensagens ou remover commits antes da publicação.
 - **Regra de ouro:** Nunca faça rebase de branches públicas compartilhadas sem o acordo explícito da equipe, pois isso quebra o histórico para outros colaboradores.

Resolução de Conflitos

- Conflitos ocorrem quando o Git não consegue combinar alterações automaticamente (ex: duas branches alteram a mesma linha de um arquivo de formas diferentes).
- O Git insere marcadores no arquivo (<<<<<<, =====, >>>>>>) separando a versão atual da versão recebida.
- A resolução exige a edição manual do arquivo para manter a versão desejada, a remoção dos marcadores e, em seguida, a execução de **git add** e **git commit**.

Pull Requests (PRs) e Fluxos de Trabalho

- No GitHub, o Pull Request transforma uma branch em uma proposta pública de alteração, servindo como espaço para discussão, revisão de código por pares e testes.
- **Fluxos de Trabalho:**

- **Centralizado:** Trabalho direto na branch principal; ideal apenas para exercícios pequenos.
- **Git Flow:** Usa múltiplas branches permanentes (**main**, **develop**, **release**); ideal para produtos com versões numeradas.
- **GitHub Flow:** Baseado em branches curtas, PRs e uma **main** sempre publicável; ideal para projetos web e entregas frequentes.
- **GitLab Flow:** Associa branches a ambientes (ex: *staging*, *production*).

Infraestrutura e Ecossistema

- Um *remote* (como **origin**) é um apelido para um repositório externo, que pode ser acessado via protocolos como Local, SSH ou HTTPS.
- A segurança exige a proteção de chaves privadas SSH e tokens de acesso HTTPS, além da atenção para nunca versionar credenciais ou arquivos sensíveis.
- Plataformas de hospedagem agregam valor ao Git: o GitHub foca em colaboração e código aberto; o GitLab em DevOps integrado (CI/CD); e o Bitbucket na integração com o ecossistema Atlassian.

Casos Comuns de Erros

- **Perda de trabalho em *Detached HEAD*:** Fazer commits enquanto o **HEAD** aponta diretamente para um commit e depois trocar de branch sem salvar. Para evitar a perda, deve-se criar uma branch antes de sair (**git switch -c nova-branch**) ou usar o **git reflog** para recuperar o *hash* perdido.
- **Iniciar um Merge com o diretório "sujo":** Executar **git merge** possuindo alterações não salvas pode gerar confusão ou sobrescrever dados. A solução é sempre fazer commit, descartar ou usar *stash* nas alterações antes de integrar.
- **Rebase em branch pública:** Alterar o histórico de uma branch que já foi enviada ao GitHub (com rebase) afeta todos que já clonaram o repositório, gerando divergências graves de *hashes*.
- **Commit com marcadores de conflito:** Esquecer de apagar os símbolos **<<<<<<**, **=====** e **>>>>>>** após resolver uma divergência no código e comitar o arquivo com essa sujeira visual.

Tabela de Comandos

Comando	Ação	Caso Ideal de Uso PDF
git branch <nome>	Cria uma referência local no commit atual, mas não troca para ela.	Quando você quer marcar um ponto no histórico para trabalhar depois, mas precisa continuar na branch atual.

<code>git switch <nome></code>	Altera o contexto de trabalho para uma branch já existente.	Quando você precisa transitar entre branches locais previamente criadas para revisar ou continuar um trabalho.
<code>git switch -c <nome></code>	Cria uma nova branch e já alterna para ela em um único passo.	Padrão recomendado para iniciar imediatamente o desenvolvimento de uma nova funcionalidade (<i>feature</i>) ou correção (<i>fix</i>).
<code>git branch -d <nome></code>	Exclui uma branch local com segurança (apenas se já mesclada).	Para limpar o repositório local removendo branches de trabalhos que já foram finalizados e integrados via PR.
<code>git branch -D <nome></code>	Exclui uma branch local à força, ignorando se foi mesclada.	Quando você criou um experimento em uma branch, percebeu que não vai usá-lo e deseja descartar o trabalho completamente.
<code>git merge <nome></code>	Solicita que o Git incorpore commits de uma branch de origem à branch atual.	Para integrar o trabalho validado de uma branch de <i>feature</i> de volta à linha principal, preservando o histórico da bifurcação.
<code>git merge --abort</code>	Tenta restaurar o estado anterior ao início do merge.	Quando um merge gera conflitos complexos inesperados e você percebe que iniciou a operação na branch errada ou por engano.
<code>git rebase <base></code>	Reorganiza o histórico reaplicando os commits sobre um novo ponto.	Para atualizar uma branch local e privada com os commits mais recentes da <i>main</i> , mantendo a linha do tempo linear antes de publicar.

<code>git rebase -i</code> <code><ref></code>	Abre o editor para reorganizar, combinar (<i>squash</i>) ou apagar commits.	Para agrupar e limpar vários commits pequenos e confusos ("teste1", "agora vai") em um único commit claro antes de abrir um PR.
<code>git push -u</code> <code>origin</code> <code><nome></code>	Publica a branch no repositório remoto e configura o <i>upstream</i> .	Ao finalizar o trabalho inicial em uma nova branch local e precisar enviá-la ao GitHub pela primeira vez para abrir um PR.
<code>git reflog</code>	Lista os movimentos recentes do ponteiro HEAD .	Como mecanismo de emergência para localizar o <i>hash</i> de um commit perdido após sair acidentalmente de um estado <i>detached HEAD</i> .

Glossário

- **Branch:** Referência nomeada que aponta para um commit, representando uma linha de desenvolvimento isolada.
- **HEAD:** Referência especial que indica o contexto atual do repositório (geralmente aponta para uma branch).
- **Detached HEAD:** Estado em que o **HEAD** aponta diretamente para um commit, e não para uma branch.
- **Merge:** Operação que integra alterações de uma branch em outra.
- **Fast-forward:** Tipo de integração em que a branch de destino não tem commits novos divergentes, permitindo que o Git apenas avance a referência.
- **Rebase:** Reaplicação de commits sobre uma nova base, reescrevendo o histórico e criando novos *hashes*.
- **Conflito:** Situação em que o Git não consegue combinar alterações concorrentes automaticamente, exigindo intervenção humana.
- **Pull Request:** Proposta de alteração em uma branch, acompanhada de revisão de código, discussão e validação (comum no GitHub).
- **Remote:** Apelido dado a um repositório externo/hospedado (como o **origin**) com o qual o repositório local se comunica.
- **Bare repository:** Repositório sem diretório de trabalho, usado exclusivamente como destino central de **push** e **fetch** em servidores.

Quiz de Fixação: Módulo 3 - Branches, Colaboração, Integração e Infraestrutura Git

1. O que é tecnicamente uma branch no Git?

- A) Uma pasta duplicada com uma cópia independente de todos os arquivos do projeto.
- B) Um backup automático gerado pelo GitHub a cada commit.
- C) Uma referência nomeada (ponteiro) para um commit específico.
- D) Um arquivo de configuração oculto dentro da pasta `.git`.

2. Qual é o comando recomendado, por sua clareza semântica, para criar uma nova branch e já alternar o contexto de trabalho para ela em um único passo?

- A) `git branch nova-branch`
- B) `git switch -c nova-branch`
- C) `git checkout nova-branch`
- D) `git merge nova-branch`

3. De acordo com as convenções de nomenclatura de branches, qual seria o nome mais adequado para uma branch focada em criar um guia de uso?

- A) `minha-branch`
- B) `teste-guia`
- C) `docs/guia-uso`
- D) `final-documentacao`

4. O que significa o estado de "detached HEAD" no Git?

- A) O repositório local perdeu a conexão com o repositório remoto (GitHub).
- B) A branch `main` foi excluída acidentalmente.
- C) O ponteiro HEAD deixou de apontar para uma branch e passou a apontar diretamente para um commit específico.
- D) Ocorreu um conflito irreversível durante um merge.

5. Como é possível recuperar e preservar o trabalho feito (commits) enquanto o repositório estava em estado de "detached HEAD"?

- A) Executando `git merge --abort`.

B) Criando uma nova branch a partir do estado atual, por exemplo, com `git switch -c recuperacao`.

C) Usando o comando `git push --force` para enviar ao remoto.

D) Fechando e reabrindo o terminal.

6. Em que cenário o Git consegue realizar uma integração do tipo "fast-forward"?

A) Quando as branches de origem e destino alteraram a mesma linha do mesmo arquivo.

B) Quando a branch de destino não recebeu novos commits desde que a branch de origem foi criada a partir dela.

C) Apenas quando a integração é feita através de um rebase interativo.

D) Quando o comando `git push` é executado com a flag `--force`.

7. O que caracteriza um "merge de três vias" (three-way merge)?

A) A combinação de três branches diferentes simultaneamente no mesmo comando.

B) Um merge onde o Git identifica o ancestral comum e as pontas das duas branches que avançaram, criando um novo "commit de merge".

C) A exclusão de três commits antigos para alinhar o histórico de commits.

D) Um processo exclusivo do GitHub Flow que exige três revisores no Pull Request.

8. O que deve ser feito caso um merge inicie um conflito, mas você perceba que iniciou a operação por engano e deseja restaurar o estado anterior?

A) Apagar a pasta do projeto e clonar novamente.

B) Executar o comando `git merge --abort`.

C) Fazer um novo commit vazio.

D) Executar `git rebase --continue`.

9. Durante a resolução de um conflito de merge, o que a marcação <<<<<< HEAD indica em um arquivo?

A) O fim da área de conflito.

B) O início da versão do código presente na branch atual (destino).

C) O início da versão do código que está sendo trazida pela branch externa (origem).

D) Um erro de sintaxe gerado pelo editor de código.

10. Qual é o passo correto para informar ao Git que um conflito de merge em um arquivo foi resolvido manualmente?

- A) Executar `git add <nome-do-arquivo>`.
- B) Apagar os arquivos ocultos `.git`.
- C) Executar `git push origin main`.
- D) Usar o comando `git switch -c resolvido`.

11. Qual é a principal diferença de resultado no histórico entre o uso de `merge` e `rebase`?

- A) O merge reescreve os hashes, enquanto o rebase preserva o histórico exato.
- B) O rebase não permite resolver conflitos, abortando a operação automaticamente.
- C) O merge preserva as bifurcações e pontos de integração, enquanto o rebase reorganiza os commits criando uma linha contínua e recriando os hashes.
- D) O rebase é obrigatório no GitHub, enquanto o merge só funciona localmente.

12. Qual é a "regra de ouro" (ética de reescrita) ao utilizar o comando `git rebase`?

- A) Sempre aplicar rebase antes de qualquer commit inicial.
- B) Não fazer rebase de branches públicas e compartilhadas sem o acordo explícito da equipe.
- C) Utilizar rebase apenas para mesclar arquivos binários.
- D) O rebase deve ser usado para substituir a branch `main` semanalmente.

13. No contexto do comando `git rebase -i` (rebase interativo), para que serve a ação `squash`?

- A) Para excluir um commit do histórico permanentemente.
- B) Para alterar apenas a mensagem de um commit anterior.
- C) Para combinar (mesclar) pequenos commits em um único commit mais coeso.
- D) Para pausar o rebase e testar o código.

14. No GitHub, qual é o papel principal de um "Pull Request" (PR)?

- A) Fazer o download compulsório de todo o código de um repositório.
- B) Transformar uma branch local não publicada na branch `main`.

C) Criar um espaço estruturado para revisão, discussão, verificação e proposta de integração de alterações.

D) Substituir a necessidade de usar o terminal Git.

15. Qual opção representa uma estratégia de integração de Pull Request que combina todos os commits da branch de funcionalidade em um único commit na branch de destino?

A) Merge commit.

B) Squash and merge.

C) Rebase and merge.

D) Fast-forward.

16. Qual fluxo de trabalho (workflow) é caracterizado pelo uso estruturado de branches como **main (produção), **develop** (integração), **feature** (funcionalidades), **release** (versões) e **hotfix** (correções urgentes)?**

A) Fluxo Centralizado.

B) GitHub Flow.

C) Git Flow.

D) GitLab Flow.

17. O "GitHub Flow" é frequentemente recomendado para projetos de documentação ou entregas contínuas (web) devido à sua abordagem. Qual é a regra básica desse fluxo?

A) A branch **main** deve estar sempre em estado publicável; o trabalho ocorre em branches curtas que passam por revisão (PR) antes de integrarem a **main**.

B) O uso exclusivo de commits diretamente na **main**, dispensando ramificações.

C) A criação obrigatória de branches de **release** a cada semana.

D) O uso de branches específicas para cada ambiente, como **staging** e **production**.

18. O que difere o "GitLab Flow" dos demais fluxos de trabalho tradicionais?

A) Ele não permite a criação de branches de **feature**.

B) Procura equilibrar simplicidade associando branches diretamente a ambientes reais de entrega (como staging, pre-production e production).

C) Substitui o uso do comando commit por ferramentas visuais de arrastar e soltar.

D) Exige a aprovação de dez revisores para qualquer merge.

19. Na infraestrutura do Git, o que é um "remote" (como o `origin`)?

A) O nome de um protocolo de rede obsoleto.

B) Um apelido (alias) configurado localmente para referenciar um repositório hospedado em outro local (ex: GitHub, servidor ou outra pasta).

C) Uma branch remota que não pode ser copiada.

D) Uma funcionalidade exclusiva de repositórios pagos.

20. Qual é a principal característica de um "bare repository" (repositório bare)?

A) Ele não possui histórico de commits.

B) É um repositório sem diretório de trabalho (working directory) visível, projetado para atuar como destino central para operações de push e fetch.

C) É um repositório que aceita apenas arquivos de imagem e áudio.

D) É a pasta onde o GitHub armazena as senhas de acesso.

21. Qual protocolo de acesso utiliza um par de chaves (pública e privada) para garantir um canal seguro de comunicação e autenticação?

A) Local.

B) HTTP simples.

C) SSH.

D) FTP.

22. Para garantir segurança da infraestrutura do projeto, quais itens NUNCA devem ser versionados (commitados) no repositório?

A) Arquivos de documentação Markdown (`.md`).

B) Segredos, tokens de acesso, chaves privadas e arquivos de variáveis de ambiente (`.env`).

C) Scripts de automação e testes automatizados.

D) Mensagens de commit com mais de 50 caracteres.

23. Qual comando exibe uma lista das branches que já foram integradas à branch atual?

A) `git branch -v`

B) `git branch --merged`

C) `git branch -D`

D) `git status`

24. O que faz o comando `git push -u origin nome-da-branch` na sua primeira execução?

A) Apaga a branch remota e a substitui pela versão local.

B) Publica a branch local no remoto (origin) e configura o `upstream` (rastreamento) entre ambas.

C) Baixa as alterações do remoto e sobrescreve a branch local.

D) Cria um Pull Request automaticamente aprovado.

25. O que a referência relativa `HEAD~1` indica em comandos Git?

A) O commit mais recente da branch `main`.

B) O primeiro commit já feito na história do repositório.

C) O commit ancestral (pai) imediato do commit atual apontado pelo HEAD.

D) Uma branch que foi deletada localmente.

26. Ao clonar um repositório, o Git cria referências de rastreamento remoto, como `origin/main`. O que essa referência representa?

A) Uma branch local que pode ser editada diretamente sem precisar de commit.

B) A última posição conhecida da branch `main` no repositório remoto `origin` após a sincronização (fetch/clone).

C) Um backup oculto criado a cada comando `git status`.

D) Um erro indicando que o clone falhou.

27. Ao realizar um fork de um projeto no GitHub para o seu perfil e cloná-lo, por que é recomendável a criação de uma nova branch antes de começar a codificar?

A) Porque branches principais não aceitam alterações de usuários sem conta premium.

B) Porque isola as alterações em uma unidade compreensível, facilita a revisão no pull request e mantém sua `main` alinhada com a versão original.

C) Porque o Git deleta o projeto após 30 dias se apenas a branch `main` for utilizada.

D) Porque branches extras reduzem o tamanho final do repositório em disco.

28. Se um arquivo binário (como uma imagem) sofrer conflito de merge, por que a resolução é mais complexa do que em um arquivo de texto?

- A) Porque o Git não consegue mesclar arquivos binários linha a linha, exigindo a escolha completa de uma versão sobre a outra.
- B) Porque arquivos binários automaticamente corrompem o ponteiro HEAD.
- C) Porque o GitHub bloqueia repositórios com conflitos binários.
- D) Porque imagens não têm permissão para entrar no `.gitignore`.

29. O utilitário `git-shell` é empregado em servidores Git por qual razão primária de segurança?

- A) Para fornecer interface gráfica a usuários de Linux.
- B) Para atuar como um shell restrito, permitindo operações Git (como push/pull) sem conceder acesso total ao sistema do servidor.
- C) Para burlar firewalls de rede corporativos.
- D) Para limpar o histórico de commits automaticamente toda semana.

30. Considerando as particularidades das plataformas de hospedagem, qual destas plataformas se destaca pelo seu foco inicial e ferramentas fortemente orientadas a DevOps integrado e opções de auto-hospedagem (self-managed)?

- A) GitHub
- B) GitWeb
- C) Bitbucket
- D) GitLab

Gabarito e Justificativas

1. **C** - Uma branch não copia os arquivos, mas cria ou atualiza um ponteiro leve indicando qual commit representa a ponta daquela linha de desenvolvimento.
2. **B** - A opção `-c` no comando `git switch` permite criar a branch e imediatamente trocar o contexto do ponteiro HEAD para ela, reduzindo a carga cognitiva por sua clareza semântica.
3. **C** - Nomes descritivos usando prefixos como `docs/`, `feature/` e `fix/` ajudam a identificar rapidamente a natureza e o escopo da alteração.
4. **C** - Detached HEAD acontece quando o HEAD perde a relação simbólica com o nome de uma branch e passa a apontar de forma direta e exclusiva para o hash de um commit.
5. **B** - O uso de `git switch -c <nome>` cria uma branch nova apontando para o commit onde o HEAD destacado está, preservando o contexto e os commits criados.

6. **B** - O "fast-forward" ocorre quando o Git constata que não há divergência histórica; a branch de destino não avançou, logo, o ponteiro apenas se move para frente até alcançar o commit da origem.
7. **B** - Quando ambas as linhas avançaram paralelamente, o Git compara o último ancestral comum e as duas pontas, unificando as mudanças através de um novo commit com dois pais (merge de três vias).
8. **B** - Se uma integração for indesejada após o disparo de conflitos, o comando `git merge --abort` aborta o processo e tenta restaurar o repositório para o estado exatamente anterior ao comando de merge.
9. **B** - O marcador `<<<<<< HEAD` identifica de forma visual no código o início do bloco onde se encontra a versão que já pertencia à branch de destino.
10. **A** - Após o desenvolvedor realizar as edições semânticas e apagar os delimitadores de conflito visual, deve-se usar o `git add` no arquivo para sinalizar ao Git que aquela pendência foi resolvida.
11. **C** - Enquanto o merge explicita a convergência criando um histórico que preserva a existência paralela de branches, o rebase altera a base dos commits da branch atual recriando a cronologia linear, o que ocasiona a substituição dos hashes.
12. **B** - Alterar e reescrever o histórico (função do rebase) de ramificações que outros colaboradores já baixaram e dependem pode gerar sérias divergências e perdas de referência para a equipe.
13. **C** - A opção `squash` na janela de rebase interativo permite juntar vários pequenos commits sequenciais em um commit singular mais limpo e legível.
14. **C** - Um Pull Request transcende um pedido técnico de mesclagem, funcionando como um ambiente oficial de revisão por pares, discussões sobre o código, auditorias automatizadas e registro de decisões arquiteturais.
15. **B** - A estratégia "Squash and merge" comprime todo o histórico desenvolvido ao longo da branch de funcionalidade transformando-o em um só commit no momento em que aterrissa na `main`.
16. **C** - O modelo Git Flow define estruturas rígidas e propósitos separados para ramificações fixas e temporárias, sendo particularmente recomendado para ecossistemas de software com ciclos rigorosos e versionamentos fixos de releases.
17. **A** - O GitHub Flow orienta que todo experimento ocorra em branches temporárias e curtas; estas são integradas repetidamente na `main` (por PR), garantindo que a base do projeto esteja imaculada e sempre apta a deploy.
18. **B** - O modelo GitLab Flow acrescenta ramificações voltadas a refletir ambientes reais da infraestrutura de hospedagem da empresa (como homologação e produção), criando um espelho lógico do ambiente de entrega.
19. **B** - Um "remote" como o origin é simplesmente uma string (apelido) configurada no repositório para apontar para a URL de um repositório Git hospedado externamente.
20. **B** - Por não conterem uma árvore de diretórios de trabalho e edição (`working directory`), os "bare repositories" atuam eficientemente como centros de recepção e sincronização de dados para equipes.
21. **C** - O protocolo SSH recorre à criptografia por meio de uma chave pública (alocada no servidor/GitHub) pareada com uma chave privada (salva e protegida na máquina local) para acesso seguro.
22. **B** - O armazenamento público ou compartilhado de tokens de nuvem, senhas ou arquivos `.env` compromete totalmente a segurança, e apagar isso em commits posteriores pode não ocultá-los por completo do histórico do projeto.

23. **B** - A tag `--merged` restringe a listagem de branches apenas àquelas cujos commits já se encontram integralmente contidos no histórico da branch atual em que o comando é rodado.
24. **B** - A adição do parâmetro `-u` (upstream) durante o primeiro envio de uma branch cria a vinculação persistente entre a sua versão local e a réplica armazenada no servidor remoto designado.
25. **C** - Expressões de histórico relativas como `HEAD~1` servem como atalhos diretos para denotar o ancestral exato (commit pai) do qual derivou o commit da posição corrente do ponteiro.
26. **B** - Ramificações marcadas como remotos (ex: `origin/main`) não permitem desenvolvimento direto sem estado *detached*; elas funcionam apenas como um marcador ou "foto" visualizando o ponto em que o servidor remoto estava na última sincronização com a sua máquina local.
27. **B** - O isolamento de mudanças em ramificações dedicadas é uma boa prática universal; ela propicia revisões bem delimitadas e preserva a organização histórica comparativa sem prejudicar a principal.
28. **A** - Diferente do processamento textual linha a linha que permite mesclagens simultâneas por região, o Git interpreta binários em bloco fechado, logo é impossível gerar uma combinação semântica mista entre duas imagens.
29. **B** - A implementação de usuários e acessos controlados através de shells restritos como o `git-shell` protege a máquina hospedeira de injeções de comandos operacionais nocivos, limitando as interações apenas ao controle de versão.
30. **D** - Enquanto o GitHub centraliza seu destaque no portfólio colaborativo open source, a plataforma GitLab foi nativamente pensada oferecendo recursos enraizados em esteiras robustas de Integração/Entrega Contínuas (CI/CD) e opções autogerenciadas locais.